



Projet Zeldiablo

Sommaire

01 : Introduction

02 : Présentation différentes versions

03 : Présentation de la conception final

04 : Présentation points intéressants

05 : Conclusion

INTRODUCTION

Partie 1:

But :

- gagner en récupérant l'amulette
- ne pas se faire tuer par les monstres

Partie 2:

Démonstration au tableau par Marius

Présentation différentes versions

version 1 :

- création du Labyrinthe
- création de la classe perso

version 2 :

- création de la classe monstre
- création de collision
- création du fait que le personnage et les monstres peuvent mourir
- et fin du tp de préparation

Présentation différentes versions

version 3 :

- Déplacement du monstre
- système d'Attaque du joueur

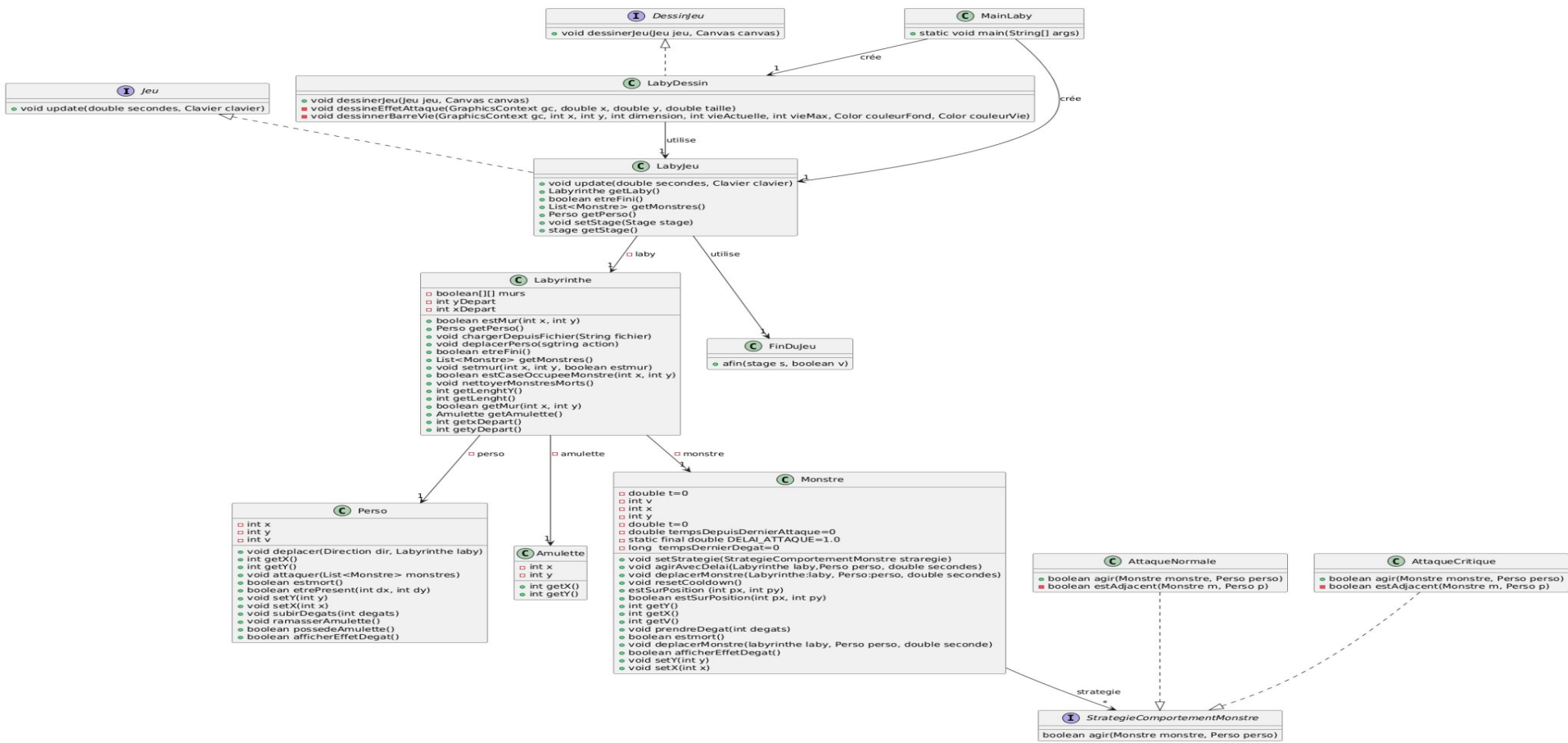
version 4 :

- Mise en place de l'amulette
- système d'Acquisition de l'amulette
- Victoire du héros/fin du jeu
- système d'Attaque des monstres

version 5 :

- Affichage des sprites (héros, monstres, amulette)
- Affichage d'un cercle rouge lors d'une attaque
- Ajout d'une barre de vie pour les personnages
- Ajout d'une fenêtre de fin de jeu

Présentation de la conception finale



Présentation de la conception finale

Résumé répartition des tâches:

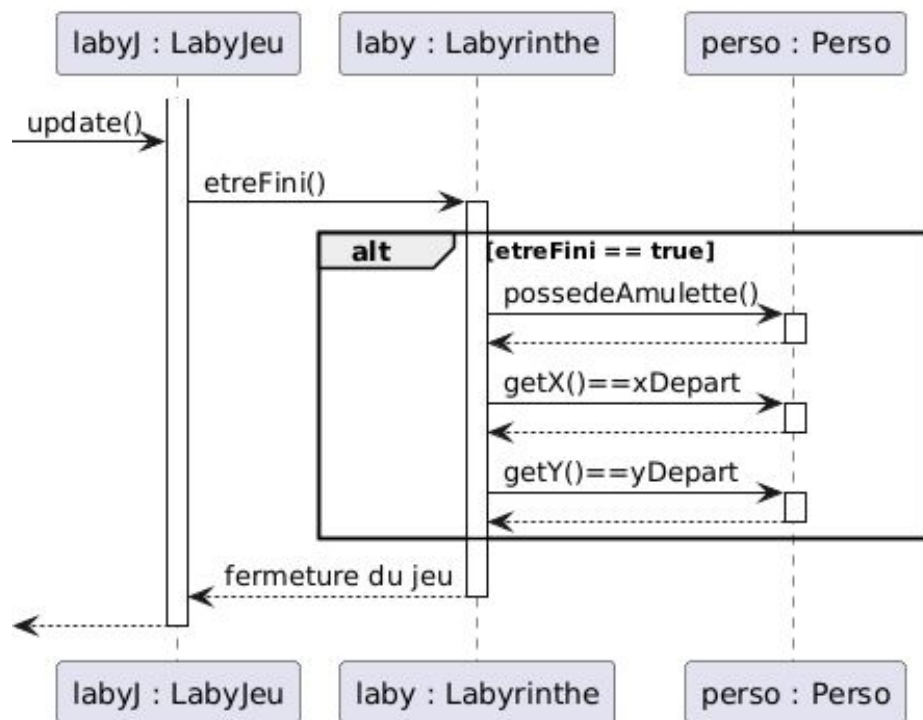
- Cyprien: diagramme de séquence et de classe ,plus autre méthode
- Arnaud: attaquer perso, poser amulette, fin du jeu avec l'amulette ,plus autre méthode
- Marius: tp partie 2 , stratégie attaquer monstre,plus autre méthode
- Fahim: déplacer monstre, barre de vie, sprite, effet attaquer, ramasser amulette, fenêtre fin de jeu défaite ou victoire, plus autre méthode

Présentation points intéressants

Fin du jeu avec l'amulette :

- méthode `etreFini`
- vérification :
 - le personnage possède l'amulette (méthode `possedeAmulette`)
 - il est aux coordonnées de départ

Diagramme de sequence etreFini(ecran de fin)



Présentation points intéressants

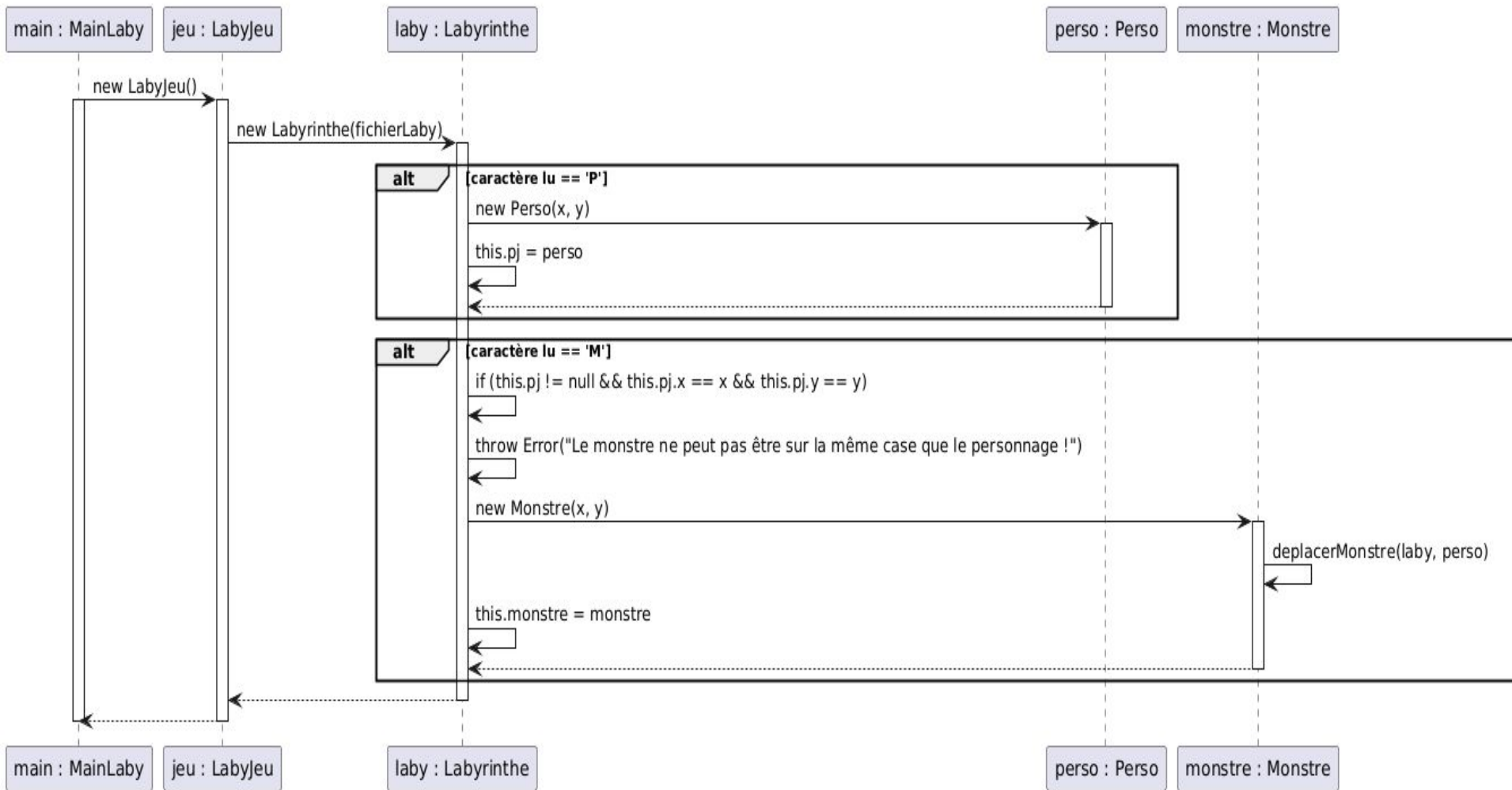
déplacer monstre aléatoirement:

- délai de 0.5 secondes
- choix entre une direction: haut,bas, gauche, droite
- vérification:
 - que ce n'est pas mûr,
 - que ce n'est pas occupé par un monstre ou perso,
 - que le monstre pas mort
- mettre les monstres à leur nouvelle position

Petit difficulté: confusion du point de départ

```
int[][] directions = {  
    { 0, -1 }, // haut  
    { 0, 1 }, // bas  
    { -1, 0 }, // gauche  
    { 1, 0 } // droite  
};
```

Diagramme de sequence premiere fonctionnalitee deplacerMonstre



Présentation points intéressants

présentation du ramassage de l'amulette

Point intéressant car :

- fin du jeu
- complexe



Diagramme de sequence premiere fonctionnalitee ramasserAmulette

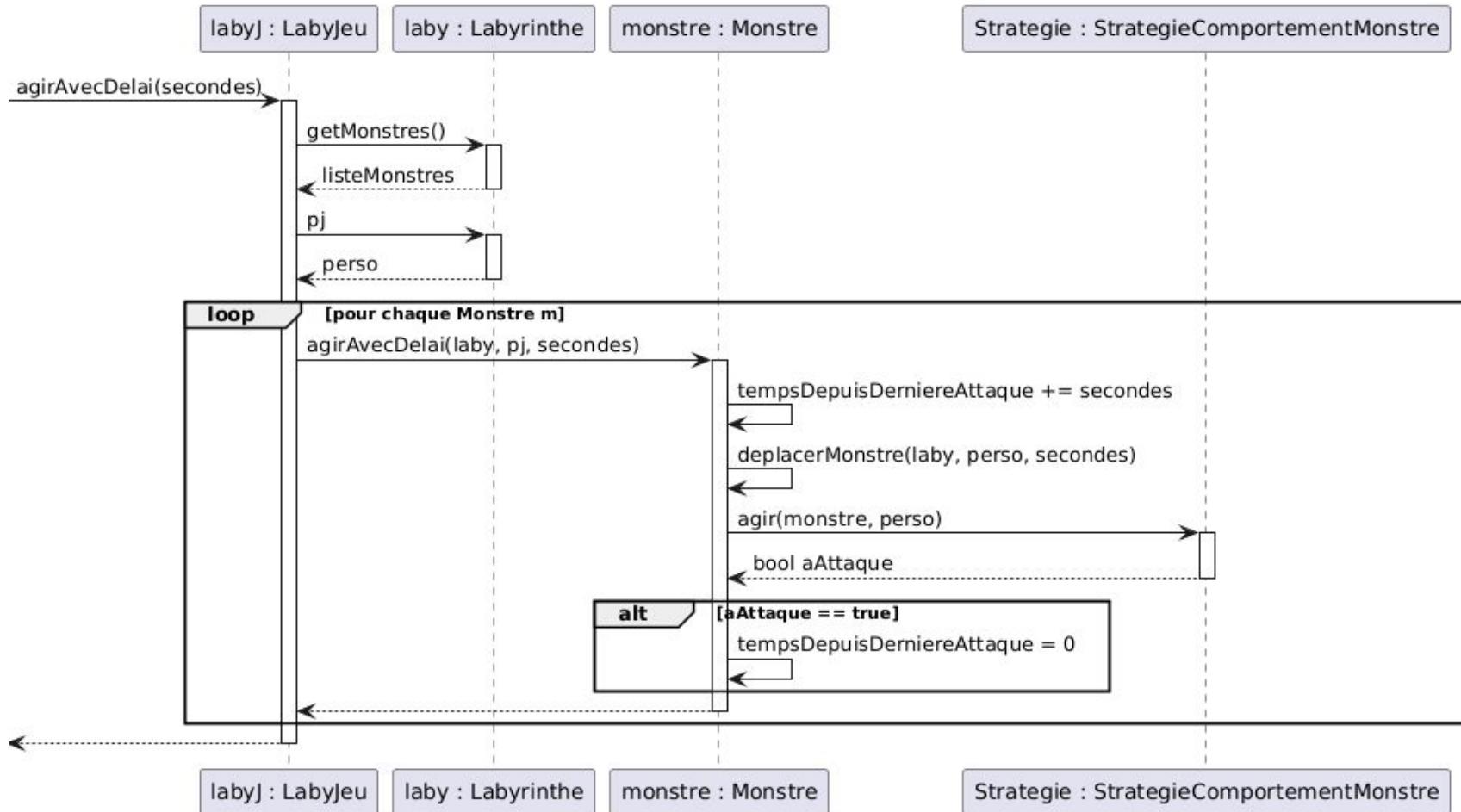


Présentation points intéressants

Pourquoi utiliser un patron Strategy ?

- Séparer le comportement d'attaque des monstres de leur structure.
- Le monstre n'intègre pas directement son comportement. Il délègue à une stratégie (StrategieComportementMonstre) qui définit la méthode agir().
- Donc au final, cela permet flexibilité (changement de comportement sans modifier la classe Monstre), lisibilité (chaque stratégie - ici AttaqueNormale et AttaqueCritique - est dans sa propre classe), extensibilité (ajout facile de nouveaux comportements si besoin), mais on peut aussi ajouter du code sans modifier l'existant.
- Dans notre jeu, deux comportements implémentés :
 - AttaqueNormale (1 dégât)
 - AttaqueCritique (1 ou 2 dégâts avec probabilité)

Diagramme de sequence troisieme fonctionnalitee attaque du Monstre



Conclusion

Les Difficultés Rencontrées